Лабораторная работа №2 — LangGraph

1. Постановка и архитектура

Дата проверки: 13 ноября 2026 года. Разработан управляемый помощник по документации ALT Linux на базе RAG ЛР1. Цель — явная маршрутизация, инструменты, цикл проверки ответа, сохранение диалога и подтверждение локального действия. Спецификация из 18 пунктов приведена в SPEC.md.

Исходная рабочая копия — /workspace/AI-lab2, remote — https://github.com/Papla2016/AI-lab2.git; исходный commit — ae3a0516d5def7b9b502541c79c8626f5c729215. До изменений рабочее дерево было чистым, ветка — work, исходные тесты: **15 passed**. HEAD исходного AI-lab1 совпадал с этим commit. GitHub API показывает fork=false для AI-lab2: это отдельная копия истории, а не зарегистрированный GitHub fork. Исходный репозиторий Papla2016/AI-lab1 не изменяется. Пользователь разрешил работу в AI-lab2; изменения подготовлены в отдельной ветке lab2-langgraph.

Новый код изолирован в src/agent/. Реально используется langgraph.graph.StateGraph; самодельного исполнителя и ReAct-агента нет. AgentTools.document_retrieval вызывает **существующий** RAGPipeline.retrieve. Dense/hybrid, BM25/RRF, top-N/final-K, reranking и исходные metadata сохраняются. Порог проверяется по dense similarity, а не по несопоставимому RRF-score. Генератор использует исходный _prompt ЛР1 и тот же интерфейс LLM.generate. CLI/Streamlit ЛР1 и исходные модули RAG не переписаны.

State и семантика обновлений

Поле	Назначение и обновление
messages	Human/AI-сообщения; Annotated[..., add_messages], накопление по ID
retrieved	Текст, полные metadata, score реально найденных чанков
draft	Сырой JSON генерации, в том числе ошибочный
critique	decision, issues, relevant, grounded последней проверки
iterations	Число входов в generate текущего запроса: 0–3
approved	Строгий boolean; true только после подтверждения нужного действия
final	Совместимое с RAGAnswer расширение AgentResult
Дополнительные	question, run_id, route, pending_action, tool_result, error

Все поля, кроме messages, используют замену значения. Узел route сбрасывает операционные поля каждого нового запроса. Память предыдущего ответа не становится документальным доказательством. Явное уточнение получает предыдущий пользовательский вопрос, затем выполняет новый retrieval. Другой thread_id начинает пустую историю.

2. Граф, цитирование и цикл критики

Полная диаграмма с условиями содержится в `graph.mermaid`.

```
START → route
  document → retrieve — usable → generate → critique
                    └ empty/weak/error → finalize
  time → call_tools → finalize
  smalltalk / out_of_scope → finalize
  side_effect → approve ⇄ HUMAN (interrupt / Command resume)
                     ├ yes → call_side_effect → finalize
                     └ no/invalid → finalize
critique: accept / fallback → finalize
          fix, iterations < 3 → generate
          fix, iterations = 3 → finalize
finalize → END
```

Маршрутизатор распознаёт точные приветствия, запрос времени и явную команду «Отправь отчёт: текст» без LLM. Остальные запросы классифицируются structured LLM-вызовом. Модель может разрешить только document; неизвестный маршрут, исполнение команд, посторонняя тема и ошибка классификации завершаются отказом. Модель не может самостоятельно вызвать инструмент с побочным эффектом.

I_max = 3. Счётчик увеличивается при каждом входе в generate, включая неудачную генерацию. Классификатор и критик его не увеличивают. Поэтому возможно не более трёх черновиков и до семи LLM-вызовов всего: один router, три generate, три семантических critique. При исправимой ошибке замечания передаются следующему generate; третья отрицательная проверка завершает граф, а не создаёт новый цикл.

Критик сначала проверяет Pydantic JSON, непустой answer и допустимость каждой точной пары `(source, chunk_id)` в CONTEXT. Достаточный ответ требует цитат; недостаточный не должен их содержать. Затем отдельная инструкция той же LLM проверяет релевантность и опору каждого утверждения на **процитированные** чанки. Структурно верная, но нерелевантная citation не гарантирует accept. Противоречивый accept преобразуется в fix. Ошибка семантической проверки даёт fallback. Текст замечаний не попадает в аудит.

Внешний формат — конверт Outcome: thread_id, run_id, status, result, interrupt. Завершённый result содержит поля RAGAnswer (`answer`, `citations`, `confidence`, `insufficient_context`) и status, iterations, tool_result. При паузе result=null, interrupt содержит проверяемые аргументы действия. Неподтверждённый ответ заменяется каноническим insufficient_context с пустыми citations; по достижении лимита status=iteration_limit. Служебные ответы имеют отдельные статусы и не выдаются за ответы из документации.

3. Инструменты, HITL и сохранение состояния

Инструмент	Аргументы	Результат и эффект
document_retrieval	question: str	Чанки ЛР1 с исходной provenance; чтение индекса
get_current_time	нет	ISO 8601 + Europe/Moscow; без побочного эффекта
send_report	action_id, body, approved	Локальная SQLite-запись фиктивной отправки

ReportAction ограничивает текст 4000 символами и не принимает пользовательский путь, получателя, shell или сетевой адрес. send_report выполняется только после approve. Внутри approve вызывается настоящий `interrupt()`; состояние сохраняется в `SqliteSaver`, а процесс CLI может завершиться. Другой процесс продолжает тот же thread через `Command(resume={approved: boolean, action_id: str})`. Строка вместо boolean, чужой ID и отказ не разрешают действие. Новый вопрос в приостановленном диалоге запрещён до решения пользователя.

Документация LangGraph предупреждает о повторном входе в узел с interrupt и повторении эффектов после сбоя перед checkpoint. Поэтому эффект находится в отдельном узле **после** approve. Локальная таблица reports с PRIMARY KEY action_id одновременно служит имитацией отправки и защитой от повторной записи. Запись и дедупликация атомарны в одной SQLite-транзакции. Отказ даже не создаёт файл отчётов. Тест аварии после commit отчёта и до checkpoint подтвердил одну запись после восстановления. Повторный resume завершённого запроса не вызывает инструмент.

Checkpoint находится в `storage/agent_checkpoints.sqlite`, отчёты — в `storage/agent_reports.sqlite`; оба исключены из Git. Это встроенный checkpointer, не переменная процесса и не Streamlit session_state. Настройки наследуют `.env` ЛР1. Команды одного thread_id должны выполняться последовательно.

```
python -m src.agent.cli ask "Привет" --thread-id demo
python -m src.agent.cli ask "Отправь отчёт: Проверка ALT Linux" --thread-id report --trace
python -m src.agent.cli status --thread-id report
python -m src.agent.cli resume --thread-id report --decision yes --trace
# Для отказа от нового запроса: --decision no
# Восстановление после сбоя без ожидающего HITL:
python -m src.agent.cli continue --thread-id report
```

JSONL-аудит `logs/graph_run.jsonl` создаётся при реальном исполнении узлов: thread_id, run_id, узел, переход, инструмент, decision, номер генерации, события HITL и итоговый status. В него не записываются вопросы, документы, тело отчёта, ключи, prompts или provider exceptions. Фикстурные прогоны имеют mode=fixture; реальные безопасные CLI-прогоны — mode=live.

API проверен по установленным версиям и официальным страницам Interrupts и Persistence. Версии: LangGraph 1.2.21, SQLite checkpointer 3.1.0, langchain-core 1.8.0.

4. Проверки, демонстрация и ограничения

Проверено на Python 3.12 / Linux. Unit-тесты используют stub LLM и настоящий StateGraph с SQLite. Интеграционные тесты выполняют реальный Chroma retrieval ЛР1 в обоих режимах и отдельные процессы CLI. Векторы/LLM для Chroma-теста детерминированы: это проверка интеграции, а не качества embedding-модели.

Набор	Наблюдаемый результат
Исходная ЛР1 до изменений	15 passed
Unit после добавления агента, включая ЛР1	49 passed
Local integration: dense, hybrid, CLI processes	3 passed
Полный набор	52 passed, 1 skipped
pip check	No broken requirements found
Внешняя LLM + пользовательский корпус	Не запускалось: корпус/индекс пусты, Ollama недоступен

Автоматически проверены корректная factual citation, слабая/пустая выдача, структурная и семантическая регенерация, I_max, ошибки генератора/критика, неизвестные sources/IDs, подтверждение/отказ/неверный action_id, восстановление после сбоя, сохранение/изоляция threads, сброс полей и безопасные маршруты. Прежние 15 тестов ЛР1 проходят без изменения. Reranker проверен spy-объектом на вызов и сохранение metadata; настоящая cross-encoder модель не загружалась.

Демо запускается `python -m evaluation.demo_agent`. Ниже — семь сценариев с **явными фикстурами** корпуса и LLM, реально исполненных через LangGraph. Fixture-guide — тестовый фрагмент, а не документ пользовательского корпуса.

Запрос / сценарий	Наблюдаемый результат
«Привет»	smalltalk, 0 генераций, без retrieval
«Напиши рецепт пирога»	refused, 0 генераций
«Сколько времени?»	time, ISO 8601 / Europe/Moscow
«Как обновить список пакетов ALT Linux?»	critique fix → generate → accept; answered, 2 генерации
Вопрос об отсутствующей функции	insufficient_context, citations=[], 0 генераций
«Отправь отчёт: демонстрация согласия» + yes	interrupt → resume, action_completed
«Отправь отчёт: демонстрация отказа» + no	interrupt → resume, action_rejected

В демо первый factual-черновик содержал неподтверждённую перезагрузку; критик запросил исправление, второй ответ использовал реальную пару ID из переданной фикстуры. Сценарии HITL закрывали и заново открывали checkpointer. Логи записаны исполнителем, а не подготовлены вручную. Обнуление корпуса и ответы LLM в этом сценарии — явные фикстуры; результаты нельзя считать реальной оценкой ALT Linux RAG.

Для полной локальной интеграции загрузите документацию ALT Linux, настройте LLM, выполните `python -m src.main index`, затем:

```
RUN_LIVE_AGENT=1 AGENT_LIVE_QUESTION="Как обновить список пакетов ALT Linux?" \
  pytest -q -m live
```

Семантический критик той же LLM может ошибаться и не доказывает истинность формально. Проверки наличия citation детерминированы. Не проверены реальные ответы внешней LLM, качество multilingual E5 и cross-encoder на ALT Linux, а также интерактивная работа Streamlit с пользовательским корпусом. Мультиагентность и новая реализация retrieval не добавлялись.